```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no, maximum-scale=1.0">
 <title>Pizzeria Listening Quest 3D - ESL Learning Game</title>
 <style>
  * { box-sizing: border-box; user-select: none; -webkit-user-select: none; margin: 0; padding: 0; }
  body, html { width: 100%; height: 100%; overflow: hidden; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: #000; color: #fff; }
  #canvas-container { width: 100%; height: 100%; position: absolute; top: 0; left: 0; z-index: 1; }

  /* Crosshair */
  #crosshair {
   position: absolute; top: 50%; left: 50%; width: 12px; height: 12px;
   margin-top: -6px; margin-left: -6px; border: 2px solid rgba(255,255,255,0.8);
   border-radius: 50%; pointer-events: none; z-index: 10; transition: transform 0.15s ease, border-color 0.15s ease;
  }
  #crosshair.active { transform: scale(1.6); border-color: #4CAF50; background-color: rgba(76, 175, 80, 0.4); }

  /* UI Overlay Header */
  #ui-header {
   position: absolute; top: 12px; left: 12px; right: 12px; z-index: 20;
   display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px; pointer-events: none;
  }
  .hud-card {
   background: rgba(20, 15, 10, 0.85); border: 2px solid #d4a359; border-radius: 12px;
   padding: 8px 16px; color: #fff; pointer-events: auto; box-shadow: 0 4px 15px rgba(0,0,0,0.5);
   backdrop-filter: blur(5px);
  }
  #mission-tracker { font-size: 0.95rem; font-weight: 600; color: #ffe8a1; }
  #mission-step { font-size: 1.1rem; color: #fff; font-weight: bold; margin-top: 2px; }

  .header-btn {
   background: #d4a359; border: none; color: #2a1805; padding: 8px 14px;
   border-radius: 8px; font-weight: bold; cursor: pointer; transition: background
```

```
0.2s, transform 0.1s;
    font-size: 0.9rem;
  }
  .header-btn:hover { background: #f0c37b; transform: translateY(-1px); }
  .header-btn:active { transform: translateY(1px); }

  /* Touch Controls */
  #joystick-zone {
    position: absolute; bottom: 30px; left: 30px; width: 120px; height: 120px;
    z-index: 30; display: none; touch-action: none;
  }
  #joystick-base {
    width: 100%; height: 100%; background: rgba(255,255,255,0.2); border: 2px
solid rgba(255,255,255,0.4);
    border-radius: 50%; position: relative;
  }
  #joystick-stick {
    width: 50px; height: 50px; background: rgba(212, 163, 89, 0.8); border-
radius: 50%;
    position: absolute; top: 35px; left: 35px; box-shadow: 0 2px 10px
rgba(0,0,0,0.3);
  }

  #touch-look-zone {
    position: absolute; top: 80px; right: 0; bottom: 0; width: 50%;
    z-index: 15; touch-action: none; display: none;
  }

  #action-btn {
    position: absolute; bottom: 35px; right: 35px; width: 90px; height: 90px;
    background: linear-gradient(135deg, #4CAF50, #2E7D32); border: 3px solid
#fff;
    border-radius: 50%; color: white; font-weight: bold; font-size: 0.85rem;
    text-align: center; display: none; justify-content: center; align-items: center;
    z-index: 30; box-shadow: 0 6px 20px rgba(0,0,0,0.6); active-transform:
scale(0.95);
    text-shadow: 0 1px 3px rgba(0,0,0,0.8);
  }

  /* Modals */
  .modal-overlay {
    position: absolute; top: 0; left: 0; width: 100%; height: 100%;
    background: rgba(0, 0, 0, 0.75); backdrop-filter: blur(6px);
    z-index: 100; display: flex; justify-content: center; align-items: center;
padding: 15px;
```

```
  }
  .modal-box {
    background: #2c1d11; border: 3px solid #d4a359; border-radius: 16px;
    max-width: 550px; width: 100%; padding: 24px; color: #fcecd5;
    box-shadow: 0 10px 30px rgba(0,0,0,0.8); max-height: 90vh; overflow-y:
auto;
  }
  .modal-box h2 { color: #f3b246; margin-bottom: 12px; font-size: 1.6rem; text-
align: center; }
  .modal-box p { font-size: 1rem; line-height: 1.5; margin-bottom: 15px; }

  .form-group { margin-bottom: 15px; text-align: left; }
  .form-group label { display: block; font-weight: bold; margin-bottom: 5px;
color: #e6c896; }
  .form-group input, .form-group select {
    width: 100%; padding: 10px; border-radius: 8px; border: 1px solid #d4a359;
    background: #1a100a; color: #fff; font-size: 1rem;
  }

  .btn-primary {
    width: 100%; background: linear-gradient(135deg, #e67e22, #d35400);
    border: none; color: white; padding: 12px; font-size: 1.1rem; font-weight: bold;
    border-radius: 10px; cursor: pointer; margin-top: 10px; transition: transform
0.1s;
  }
  .btn-primary:hover { transform: scale(1.02); }

  /* Quiz Option Buttons */
  .quiz-option {
    display: block; width: 100%; text-align: left; padding: 12px 16px;
    margin: 8px 0; background: #3d2a1b; border: 2px solid #735135;
    color: #fff; border-radius: 10px; font-size: 1rem; cursor: pointer;
    transition: all 0.2s;
  }
  .quiz-option:hover { background: #5c402b; border-color: #d4a359; }

  /* Floating Station 3D Badges */
  .station-badge {
    position: absolute; background: rgba(0, 0, 0, 0.8); color: #ffe8a1;
    padding: 4px 10px; border-radius: 20px; border: 1px solid #d4a359;
    font-size: 0.85rem; font-weight: bold; pointer-events: none; z-index: 12;
    transform: translate(-50%, -100%); transition: opacity 0.3s;
  }

  /* Ingredient Selection Palette UI */
```

```
   #topping-palette {
     position: absolute; bottom: 20px; left: 50%; transform: translateX(-50%);
     background: rgba(20, 15, 10, 0.9); border: 2px solid #d4a359; border-radius:
16px;
     padding: 10px 16px; z-index: 25; display: none; gap: 10px; align-items: center;
     box-shadow: 0 4px 20px rgba(0,0,0,0.6);
   }
   .topping-btn {
     background: #3d2a1b; border: 1px solid #a67c48; color: #fff; padding: 8px
12px;
     border-radius: 10px; cursor: pointer; font-weight: bold; font-size: 0.85rem;
     display: flex; flex-direction: column; align-items: center; gap: 4px;
   }
   .topping-btn:hover { background: #61442c; border-color: #f3b246; }

   /* Teacher Table */
   .data-table { width: 100%; border-collapse: collapse; margin-top: 15px; font-
size: 0.85rem; }
   .data-table th, .data-table td { border: 1px solid #735135; padding: 6px 8px;
text-align: left; }
   .data-table th { background: #3d2a1b; color: #f3b246; }
  </style>

  <!-- Three.js CDN -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/
three.min.js"></script>
</head>
<body>

  <div id="canvas-container"></div>
  <div id="crosshair"></div>

  <!-- UI Header -->
  <div id="ui-header">
    <div class="hud-card">
      <div id="mission-tracker">MISSION OBJECTIVE</div>
      <div id="mission-step">1. Walk to Customer Table & Listen</div>
    </div>
    <div class="hud-card" style="display: flex; gap: 12px; align-items: center;">
      <div>Score: <span id="score-val" style="color:#f3b246; font-
weight:bold;">0</span></div>
      <div>Stars: <span id="star-val" style="color:#ffd700;">⭐⭐⭐</span></
div>
      <button class="header-btn" onclick="game.replayAudio()">🔊 Replay
Voice</button>
```

```
    <button class="header-btn" onclick="game.toggleMute()">🎵 Audio On/Off</button>
    <button class="header-btn" onclick="game.openTeacherDashboard()">📊 Dashboard</button>
   </div>
  </div>

  <!-- Touch Joystick & Action Button -->
  <div id="joystick-zone"><div id="joystick-base"><div id="joystick-stick"></div></div></div>
  <div id="touch-look-zone"></div>
  <div id="action-btn">TAP TO<br>INTERACT</div>

  <!-- Ingredient Palette for Prep Counter -->
  <div id="topping-palette">
   <button class="topping-btn" onclick="game.addTopping('mozzarella')">🧀 Mozzarella</button>
   <button class="topping-btn" onclick="game.addTopping('pepperoni')">🍕 Pepperoni</button>
   <button class="topping-btn" onclick="game.addTopping('mushrooms')">🍄 Mushrooms</button>
   <button class="topping-btn" onclick="game.addTopping('olives')">🫒 Olives</button>
   <button class="topping-btn" onclick="game.addTopping('peppers')">🫑 Bell Peppers</button>
   <button class="topping-btn" onclick="game.addTopping('basil')">🌿 Basil</button>
   <button class="header-btn" style="background:#27ae60; color:#fff;" onclick="game.finishToppings()">Done Toppings ✓</button>
  </div>

  <!-- Modal 1: Student Setup & Profile -->
  <div id="start-modal" class="modal-overlay">
   <div class="modal-box">
    <h2>🍕 Pizzeria Listening Quest</h2>
    <p>Welcome Chef! Practice your English listening skills by fulfilling pizza orders for hungry customers.</p>
    <div class="form-group">
     <label for="student-name">Student Full Name:</label>
     <input type="text" id="student-name" placeholder="e.g. Alex Tan" value="Student 1">
    </div>
    <div class="form-group">
     <label for="student-class">Class / Section ID:</label>
     <input type="text" id="student-class" placeholder="e.g. Year 4 Primary"
```

```
value="Year 4 Blue">
    </div>
    <div class="form-group">
     <label for="difficulty-select">Proficiency Difficulty Level:</label>
     <select id="difficulty-select">
      <option value="beginner">Beginner (Slow Speech, On-screen Subtitles)</
option>
      <option value="intermediate" selected>Intermediate (Standard Speech,
Partial Subtitles)</option>
      <option value="advanced">Advanced (Realistic Speed, Audio Only!)</
option>
     </select>
    </div>
    <button class="btn-primary" onclick="game.startGame()">Start Game
Mission 🚀</button>
   </div>
  </div>

  <!-- Modal 2: Listening & Order Dialog -->
  <div id="order-modal" class="modal-overlay" style="display:none;">
   <div class="modal-box">
    <h2>🗣️ Customer Order</h2>
    <p id="order-speech-text" style="font-style: italic; background:#1a100a;
padding:12px; border-radius:8px; border-left:4px solid #f3b246;">
     "Click listen below to hear my order!"
    </p>
    <button class="btn-primary" style="background:#2980b9;"
onclick="game.playCustomerAudio()">🔊 Listen to Order</button>
    <button class="btn-primary" style="background:#27ae60; margin-top:8px;"
onclick="game.closeOrderModal()">Got It! Start Cooking 🍕</button>
   </div>
  </div>

  <!-- Modal 3: ESL Comprehension Check Quiz Modal -->
  <div id="quiz-modal" class="modal-overlay" style="display:none;">
   <div class="modal-box">
    <h2>📝 ESL Listening Comprehension Quiz</h2>
    <p id="quiz-question-text" style="font-weight:bold; font-size:1.1rem;
color:#f3b246;"></p>
    <div id="quiz-options-container"></div>
   </div>
  </div>

  <!-- Modal 4: Level Complete Screen -->
  <div id="complete-modal" class="modal-overlay" style="display:none;">
```

```
  <div class="modal-box" style="text-align:center;">
    <h2>🎉 Order Delivered Successfully!</h2>
    <p>Excellent listening comprehension skills!</p>
    <div style="font-size: 2.5rem; margin: 15px 0;" id="complete-
stars">⭐⭐⭐</div>
    <p>Score Earned: <span id="complete-score" style="color:#f3b246; font-
weight:bold;">100</span> points</p>
    <button class="btn-primary" onclick="game.nextOrder()">Next Customer
Order ➡️</button>
  </div>
 </div>

 <!-- Modal 5: Teacher Dashboard & Data Export -->
 <div id="teacher-modal" class="modal-overlay" style="display:none;">
  <div class="modal-box" style="max-width:700px;">
    <h2>📊 Student Performance Dashboard</h2>
    <div class="form-group">
      <label for="webhook-url">Google Apps Script Webhook API URL (Optional
Direct Sync):</label>
      <input type="text" id="webhook-url" placeholder="https://
script.google.com/macros/s/.../exec">
    </div>
    <div style="max-height: 200px; overflow-y: auto;">
      <table class="data-table">
        <thead>
          <tr>
            <th>Time</th>
            <th>Name</th>
            <th>Level</th>
            <th>Score</th>
            <th>Accuracy</th>
            <th>Replays</th>
          </tr>
        </thead>
        <tbody id="teacher-table-body">
          <!-- Data Rows inserted dynamically -->
        </tbody>
      </table>
    </div>
    <div style="display:flex; gap:10px; margin-top:15px;">
      <button class="btn-primary" style="background:#27ae60;"
onclick="game.exportCSV()">📥 Download CSV</button>
      <button class="btn-primary" style="background:#8e44ad;"
onclick="game.syncToGoogleSheets()">☁️ Sync to Google Sheets</button>
      <button class="btn-primary" style="background:#7f8c8d;"
```

```
onclick="document.getElementById('teacher-
modal').style.display='none'">Close</button>
    </div>
   </div>
  </div>

  <script>
   /* Global Game Engine */
   class PizzeriaGame {
    constructor() {
     this.studentName = "Student 1";
     this.studentClass = "Year 4";
     this.difficulty = "intermediate";
     this.score = 0;
     this.stars = 3;
     this.audioReplayCount = 0;

     // Mission Workflow: 1 = Order, 2 = Dough & Sauce, 3 = Toppings, 4 = Baking,
5 = Serve & Quiz
     this.currentStep = 1;

     this.currentOrder = null;
     this.preparedPizza = { dough: false, sauce: false, toppings: [] };
     this.gameLogs = [];

     // Dynamic Orders Bank
     this.ordersPool = [
      {
       speech: "Hello Chef! I would like a pizza with tomato sauce, mozzarella
cheese, and pepperoni please!",
       sauce: "tomato",
       toppings: ["mozzarella", "pepperoni"],
       questions: [
        { q: "What main topping did the customer ask for?", options:
["Pepperoni", "Mushrooms", "Olives"], answer: 0 },
        { q: "Which sauce was requested?", options: ["Garlic Butter", "Tomato
Sauce", "BBQ Sauce"], answer: 1 }
       ]
      },
      {
       speech: "Hi! I want a vegetarian pizza with tomato sauce, mozzarella,
mushrooms, bell peppers, and basil!",
       sauce: "tomato",
       toppings: ["mozzarella", "mushrooms", "peppers", "basil"],
       questions: [
```

```
          { q: "Was pepperoni requested in this order?", options: ["Yes", "No"], answer: 1 },
          { q: "Which vegetable topping was asked for?", options: ["Mushrooms and Bell Peppers", "Pineapple", "Onions"], answer: 0 }
         ]
        },
        {
         speech: "Good day! Please make me a pizza with mozzarella cheese, olives, and mushrooms!",
         sauce: "tomato",
         toppings: ["mozzarella", "olives", "mushrooms"],
         questions: [
          { q: "How many total toppings did the customer ask for?", options: ["2 toppings", "3 toppings", "5 toppings"], answer: 1 },
          { q: "Did the customer ask for olives?", options: ["Yes", "No"], answer: 0 }
         ]
        }
       ];

       this.init3D();
       this.initAudio();
       this.initControls();
       this.setupTouchControls();
       this.animate();
      }

      /* 3D Scene & Station Rendering */
      init3D() {
       this.container = document.getElementById('canvas-container');
       this.scene = new THREE.Scene();
       this.scene.background = new THREE.Color(0x1a0f07);
       this.scene.fog = new THREE.FogExp2(0x1a0f07, 0.03);

       this.camera = new THREE.PerspectiveCamera(60, window.innerWidth / window.innerHeight, 0.1, 100);
       this.camera.position.set(0, 1.6, 3); // First-person view height

       this.renderer = new THREE.WebGLRenderer({ antialias: true });
       this.renderer.setSize(window.innerWidth, window.innerHeight);
       this.renderer.shadowMap.enabled = true;
       this.renderer.shadowMap.type = THREE.PCFSoftShadowMap;
       this.container.appendChild(this.renderer.domElement);

       // Lighting
       const ambientLight = new THREE.AmbientLight(0xfff5ea, 0.6);
```

```
    this.scene.add(ambientLight);

    const mainLight = new THREE.PointLight(0xffddaa, 1.2, 18);
    mainLight.position.set(0, 3.5, 0);
    mainLight.castShadow = true;
    this.scene.add(mainLight);

    // Brick Oven Glowing Light
    this.ovenLight = new THREE.PointLight(0xff5500, 2, 8);
    this.ovenLight.position.set(-3.5, 1.2, -3.5);
    this.scene.add(this.ovenLight);

    this.buildPizzeriaRoom();
    this.buildStations();

    window.addEventListener('resize', () => this.onWindowResize());
  }

  buildPizzeriaRoom() {
    // Checkered Floor
    const floorGeo = new THREE.PlaneGeometry(12, 12);
    const canvas = document.createElement('canvas');
    canvas.width = 256; canvas.height = 256;
    const ctx = canvas.getContext('2d');
    ctx.fillStyle = '#2c1e14'; ctx.fillRect(0,0,256,256);
    ctx.fillStyle = '#dfc299'; ctx.fillRect(0,0,128,128);
ctx.fillRect(128,128,128,128);
    const floorTex = new THREE.CanvasTexture(canvas);
    floorTex.wrapS = THREE.RepeatWrapping; floorTex.wrapT =
THREE.RepeatWrapping;
    floorTex.repeat.set(6, 6);

    const floorMat = new THREE.MeshStandardMaterial({ map: floorTex,
roughness: 0.4 });
    const floor = new THREE.Mesh(floorGeo, floorMat);
    floor.rotation.x = -Math.PI / 2;
    floor.receiveShadow = true;
    this.scene.add(floor);

    // Walls
    const wallMat = new THREE.MeshStandardMaterial({ color: 0x5c3a21,
roughness: 0.8 });

    const backWall = new THREE.Mesh(new THREE.PlaneGeometry(12, 5),
wallMat);
```

```
    backWall.position.set(0, 2.5, -6);
    this.scene.add(backWall);

    const leftWall = new THREE.Mesh(new THREE.PlaneGeometry(12, 5),
wallMat);
    leftWall.position.set(-6, 2.5, 0);
    leftWall.rotation.y = Math.PI / 2;
    this.scene.add(leftWall);

    const rightWall = new THREE.Mesh(new THREE.PlaneGeometry(12, 5),
wallMat);
    rightWall.position.set(6, 2.5, 0);
    rightWall.rotation.y = -Math.PI / 2;
    this.scene.add(rightWall);
  }

  buildStations() {
    this.stations = {};

    // 1. Customer Table & Chair
    const tableGroup = new THREE.Group();
    const tableTop = new THREE.Mesh(new THREE.CylinderGeometry(1.2, 1.2,
0.1, 16), new THREE.MeshStandardMaterial({ color: 0x8b4513 }));
    tableTop.position.y = 0.8;
    tableTop.castShadow = true;
    tableGroup.add(tableTop);

    const tableLeg = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1,
0.8, 8), new THREE.MeshStandardMaterial({ color: 0x111111 }));
    tableLeg.position.y = 0.4;
    tableGroup.add(tableLeg);

    // Customer Avatar (Simple 3D Mesh)
    const customer = new THREE.Group();
    const head = new THREE.Mesh(new THREE.SphereGeometry(0.2, 16, 16),
new THREE.MeshStandardMaterial({ color: 0xffdbac }));
    head.position.y = 1.3;
    const body = new THREE.Mesh(new THREE.CylinderGeometry(0.25, 0.3, 0.6,
16), new THREE.MeshStandardMaterial({ color: 0x2980b9 }));
    body.position.y = 0.85;
    customer.add(head); customer.add(body);
    customer.position.set(0, 0, -0.6);
    tableGroup.add(customer);

    tableGroup.position.set(3, 0, -2);
```

```
        this.scene.add(tableGroup);
        this.stations.customer = { position: tableGroup.position, name: "1. Customer
Table" };

        // 2. Prep Counter
        const prepCounter = new THREE.Group();
        const counterBase = new THREE.Mesh(new THREE.BoxGeometry(3, 0.9, 1.2),
new THREE.MeshStandardMaterial({ color: 0x3d2314 }));
        counterBase.position.y = 0.45;
        counterBase.castShadow = true;
        prepCounter.add(counterBase);

        const marbleTop = new THREE.Mesh(new THREE.BoxGeometry(3.1, 0.08,
1.3), new THREE.MeshStandardMaterial({ color: 0xdedede, roughness: 0.2 }));
        marbleTop.position.y = 0.9,
        prepCounter.add(marbleTop);

        // Interactive Pizza Base on Prep Counter
        this.pizzaBaseGroup = new THREE.Group();
        this.pizzaBaseGroup.position.set(0, 0.95, 0);

        this.doughMesh = new THREE.Mesh(new THREE.CylinderGeometry(0.35,
0.35, 0.03, 24), new THREE.MeshStandardMaterial({ color: 0xe6c896 }));
        this.doughMesh.castShadow = true;
        this.pizzaBaseGroup.add(this.doughMesh);

        this.sauceMesh = new THREE.Mesh(new THREE.CylinderGeometry(0.32,
0.32, 0.035, 24), new THREE.MeshStandardMaterial({ color: 0xc0392b }));
        this.sauceMesh.visible = false;
        this.pizzaBaseGroup.add(this.sauceMesh);

        prepCounter.add(this.pizzaBaseGroup);

        prepCounter.position.set(0, 0, -4.5);
        this.scene.add(prepCounter);
        this.stations.prep = { position: prepCounter.position, name: "2. Prep
Counter" };

        // 3. Wood-Fired Brick Oven
        const ovenGroup = new THREE.Group();
        const ovenBody = new THREE.Mesh(new THREE.BoxGeometry(2, 1.8, 1.8),
new THREE.MeshStandardMaterial({ color: 0x7f2716, roughness: 0.9 }));
        ovenBody.position.y = 0.9;
        ovenGroup.add(ovenBody);
```

```
      const ovenArch = new THREE.Mesh(new THREE.CylinderGeometry(0.5, 0.5, 0.4, 16, 1, false, 0, Math.PI), new THREE.MeshStandardMaterial({ color: 0x222222 }));
      ovenArch.rotation.x = Math.PI / 2;
      ovenArch.position.set(0, 0.8, 0.85);
      ovenGroup.add(ovenArch);

      ovenGroup.position.set(-4, 0, -3.5);
      ovenGroup.rotation.y = Math.PI / 4;
      this.scene.add(ovenGroup);
      this.stations.oven = { position: ovenGroup.position, name: "3. Wood Oven" };
    }

    /* Audio & Speech Synthesizer */
    initAudio() {
      this.audioCtx = new (window.AudioContext || window.webkitAudioContext)();
      this.isMuted = false;
    }

    playTone(freq, duration, type='sine') {
      if (this.isMuted) return;
      try {
        const osc = this.audioCtx.createOscillator();
        const gain = this.audioCtx.createGain();
        osc.type = type;
        osc.frequency.value = freq;
        gain.gain.setValueAtTime(0.15, this.audioCtx.currentTime);
        gain.gain.exponentialRampToValueAtTime(0.001, this.audioCtx.currentTime + duration);
        osc.connect(gain);
        gain.connect(this.audioCtx.destination);
        osc.start();
        osc.stop(this.audioCtx.currentTime + duration);
      } catch(e) {}
    }

    playSizzleSound() {
      if (this.isMuted) return;
      this.playTone(180, 0.8, 'sawtooth');
    }

    playSuccessSound() {
      if (this.isMuted) return;
      this.playTone(523.25, 0.15); // C5
      setTimeout(() => this.playTone(659.25, 0.15), 150); // E5
```

```
    setTimeout(() => this.playTone(783.99, 0.3), 300); // G5
  }

  toggleMute() {
    this.isMuted = !this.isMuted;
    alert(this.isMuted ? "Audio Muted" : "Audio Unmuted");
  }

  speakText(text) {
    if ('speechSynthesis' in window) {
      window.speechSynthesis.cancel();
      const utterance = new SpeechSynthesisUtterance(text);
      utterance.lang = 'en-US';

      if (this.difficulty === 'beginner') utterance.rate = 0.8;
      else if (this.difficulty === 'intermediate') utterance.rate = 1.0;
      else if (this.difficulty === 'advanced') utterance.rate = 1.2;

      window.speechSynthesis.speak(utterance);
    }
  }

  /* Touch & Keyboard Controls */
  initControls() {
    this.keys = {};
    this.moveVector = { x: 0, z: 0 };
    this.pitch = 0; this.yaw = 0;

    window.addEventListener('keydown', (e) => this.keys[e.key.toLowerCase()]
= true);
    window.addEventListener('keyup', (e) => this.keys[e.key.toLowerCase()] =
false);

    // Interaction Key
    window.addEventListener('keydown', (e) => {
      if (e.key.toLowerCase() === 'e') this.handleInteraction();
    });

    // Mouse Look
    let isDragging = false;
    document.addEventListener('mousedown', () => isDragging = true);
    document.addEventListener('mouseup', () => isDragging = false);
    document.addEventListener('mousemove', (e) => {
      if (isDragging || document.pointerLockElement === document.body) {
        this.yaw -= e.movementX * 0.003;
```

```
        this.pitch -= e.movementY * 0.003;
        this.pitch = Math.max(-Math.PI/3, Math.min(Math.PI/3, this.pitch));
      }
    });
  }

  setupTouchControls() {
    if ('ontouchstart' in window || navigator.maxTouchPoints > 0) {
      document.getElementById('joystick-zone').style.display = 'block';
      document.getElementById('touch-look-zone').style.display = 'block';
      document.getElementById('action-btn').style.display = 'flex';

      // Virtual Joystick Setup
      const stick = document.getElementById('joystick-stick');
      const base = document.getElementById('joystick-base');
      let touchId = null;

      base.addEventListener('touchstart', (e) => {
        touchId = e.changedTouches[0].identifier;
      });

      base.addEventListener('touchmove', (e) => {
        for (let t of e.changedTouches) {
          if (t.identifier === touchId) {
            const rect = base.getBoundingClientRect();
            const centerX = rect.left + rect.width / 2;
            const centerY = rect.top + rect.height / 2;
            let dx = t.clientX - centerX;
            let dy = t.clientY - centerY;
            const dist = Math.min(40, Math.hypot(dx, dy));
            const angle = Math.atan2(dy, dx);

            stick.style.transform = `translate(${Math.cos(angle)*dist}px, ${Math.sin(angle)*dist}px)`;
            this.moveVector.x = (Math.cos(angle) * dist) / 40;
            this.moveVector.z = (Math.sin(angle) * dist) / 40;
          }
        }
      });

      const resetJoystick = () => {
        stick.style.transform = `translate(0px, 0px)`;
        this.moveVector.x = 0; this.moveVector.z = 0;
      };
      base.addEventListener('touchend', resetJoystick);
```

```
        base.addEventListener('touchcancel', resetJoystick);

        // Touch Look Zone
        let lookTouchId = null, lastTouchX = 0, lastTouchY = 0;
        const lookZone = document.getElementById('touch-look-zone');
        lookZone.addEventListener('touchstart', (e) => {
          lookTouchId = e.changedTouches[0].identifier;
          lastTouchX = e.changedTouches[0].clientX;
          lastTouchY = e.changedTouches[0].clientY;
        });
        lookZone.addEventListener('touchmove', (e) => {
          for (let t of e.changedTouches) {
            if (t.identifier === lookTouchId) {
              const dx = t.clientX - lastTouchX;
              const dy = t.clientY - lastTouchY;
              this.yaw -= dx * 0.005;
              this.pitch -= dy * 0.005;
              this.pitch = Math.max(-Math.PI/3, Math.min(Math.PI/3, this.pitch));
              lastTouchX = t.clientX; lastTouchY = t.clientY;
            }
          }
        });

        // Action Button
        document.getElementById('action-btn').addEventListener('click', () =>
this.handleInteraction());
      }
    }

    /* Workflow & Mission Mechanics */
    startGame() {
      this.studentName = document.getElementById('student-name').value ||
"Student 1";
      this.studentClass = document.getElementById('student-class').value || "Year
4";
      this.difficulty = document.getElementById('difficulty-select').value;

      document.getElementById('start-modal').style.display = 'none';
      this.loadNewOrder();
    }

    loadNewOrder() {
      this.currentOrder = this.ordersPool[Math.floor(Math.random() *
this.ordersPool.length)];
      this.currentStep = 1;
```

```
    this.preparedPizza = { dough: false, sauce: false, toppings: [] };
    this.audioReplayCount = 0;

    // Reset Pizza Visual Mesh
    this.doughMesh.scale.set(1, 1, 1);
    this.sauceMesh.visible = false;

    // Clear Topping Visuals
    for (let i = this.pizzaBaseGroup.children.length - 1; i >= 2; i--) {
      this.pizzaBaseGroup.remove(this.pizzaBaseGroup.children[i]);
    }

    this.updateMissionUI("1. Walk to Customer Table & Listen");
  }

  updateMissionUI(text) {
    document.getElementById('mission-step').innerText = text;
  }

  handleInteraction() {
    const p = this.camera.position;

    // Step 1: Customer Order
    if (this.currentStep === 1 && p.distanceTo(this.stations.customer.position) <
2.5) {
      document.getElementById('order-modal').style.display = 'flex';

      if (this.difficulty === 'advanced') {
        document.getElementById('order-speech-text').innerText = "🔊 [Audio
Only - Listen Carefully!]";
      } else {
        document.getElementById('order-speech-text').innerText = `"$
{this.currentOrder.speech}"`;
      }
      this.playCustomerAudio();
    }
    // Step 2 & 3: Prep Counter
    else if ((this.currentStep === 2 || this.currentStep === 3) &&
p.distanceTo(this.stations.prep.position) < 2.5) {
      if (!this.preparedPizza.dough) {
        this.preparedPizza.dough = true;
        this.doughMesh.scale.set(1.3, 0.5, 1.3); // Knead/flatten dough animation
        this.playTone(200, 0.2, 'square');
        alert("Dough flattened and kneaded!");
      } else if (!this.preparedPizza.sauce) {
```

```
        this.preparedPizza.sauce = true;
        this.sauceMesh.visible = true;
        this.playTone(300, 0.2, 'sine');
        alert("Tomato sauce spread on pizza base!");
        this.currentStep = 3;
        this.updateMissionUI("3. Select Toppings at Prep Counter");
        document.getElementById('topping-palette').style.display = 'flex';
      }
     }
     // Step 4: Wood Oven
     else if (this.currentStep === 4 && p.distanceTo(this.stations.oven.position) <
2.5) {
      this.updateMissionUI("4. Baking Pizza in Wood Oven...");
      this.playSizzleSound();
      setTimeout(() => {
       this.playSuccessSound();
       alert("Pizza cooked to golden brown perfection!");
       this.currentStep = 5;
       this.updateMissionUI("5. Serve Customer & Take Listening Quiz");
      }, 2500);
     }
     // Step 5: Serve Customer
     else if (this.currentStep === 5 &&
p.distanceTo(this.stations.customer.position) < 2.5) {
      this.openQuizModal();
     }
    }

    playCustomerAudio() {
     this.audioReplayCount++;
     this.speakText(this.currentOrder.speech);
    }

    replayAudio() {
     if (this.currentOrder) {
      this.playCustomerAudio();
     }
    }

    closeOrderModal() {
     document.getElementById('order-modal').style.display = 'none';
     this.currentStep = 2;
     this.updateMissionUI("2. Knead Dough & Spread Sauce at Prep Counter");
    }
```

```
    addTopping(type) {
      if (!this.preparedPizza.toppings.includes(type)) {
        this.preparedPizza.toppings.push(type);

        // Visual Topping Mesh
        let topColor = 0xffffff;
        if (type === 'pepperoni') topColor = 0xc0392b;
        else if (type === 'mushrooms') topColor = 0xd5c8b8;
        else if (type === 'olives') topColor = 0x2c3e50;
        else if (type === 'peppers') topColor = 0x27ae60;
        else if (type === 'basil') topColor = 0x2ecc71;

        const topMesh = new THREE.Mesh(
          new THREE.CylinderGeometry(0.05, 0.05, 0.02, 8),
          new THREE.MeshStandardMaterial({ color: topColor })
        );
        topMesh.position.set((Math.random()-0.5)*0.4, 0.04,
(Math.random()-0.5)*0.4);
        this.pizzaBaseGroup.add(topMesh);
        this.playTone(400, 0.1);
      }
    }

    finishToppings() {
      document.getElementById('topping-palette').style.display = 'none';
      this.currentStep = 4;
      this.updateMissionUI("4. Put Pizza into Wood-Fired Oven");
    }

    /* Quiz & Evaluation System */
    openQuizModal() {
      const quizModal = document.getElementById('quiz-modal');
      const qContainer = document.getElementById('quiz-options-container');
      const qText = document.getElementById('quiz-question-text');

      this.currentQuestionIdx = 0;
      this.quizCorrectCount = 0;

      this.renderQuizQuestion();
      quizModal.style.display = 'flex';
    }

    renderQuizQuestion() {
      const qData = this.currentOrder.questions[this.currentQuestionIdx];
      document.getElementById('quiz-question-text').innerText = `Question $
```

```
{this.currentQuestionIdx + 1}: ${qData.q}`;

    const qContainer = document.getElementById('quiz-options-container');
    qContainer.innerHTML = '';

    qData.options.forEach((opt, idx) => {
      const btn = document.createElement('button');
      btn.className = 'quiz-option';
      btn.innerText = opt;
      btn.onclick = () => this.handleAnswer(idx);
      qContainer.appendChild(btn);
    });
  }

  handleAnswer(selectedIdx) {
    const qData = this.currentOrder.questions[this.currentQuestionIdx];
    if (selectedIdx === qData.answer) {
      this.quizCorrectCount++;
      this.playTone(600, 0.15);
    } else {
      this.playTone(150, 0.2, 'sawtooth');
    }

    this.currentQuestionIdx++;
    if (this.currentQuestionIdx < this.currentOrder.questions.length) {
      this.renderQuizQuestion();
    } else {
      document.getElementById('quiz-modal').style.display = 'none';
      this.finishOrder();
    }
  }

  finishOrder() {
    const totalQ = this.currentOrder.questions.length;
    const accuracy = Math.round((this.quizCorrectCount / totalQ) * 100);
    const points = accuracy * 2;

    this.score += points;
    document.getElementById('score-val').innerText = this.score;

    // Log Data
    const logEntry = {
      time: new Date().toLocaleTimeString(),
      name: this.studentName,
      class: this.studentClass,
```

```
      level: this.difficulty,
      score: points,
      accuracy: `${accuracy}%`,
      replays: this.audioReplayCount
    };
    this.gameLogs.push(logEntry);

    // UI Complete Modal
    document.getElementById('complete-score').innerText = points;
    document.getElementById('complete-stars').innerText = accuracy >= 80 ?
'⭐⭐⭐' : accuracy >= 50 ? '⭐⭐' : '⭐';
    document.getElementById('complete-modal').style.display = 'flex';
    this.playSuccessSound();
  }

  nextOrder() {
    document.getElementById('complete-modal').style.display = 'none';
    this.loadNewOrder();
  }

  /* Teacher Dashboard & Data Export */
  openTeacherDashboard() {
    const tbody = document.getElementById('teacher-table-body');
    tbody.innerHTML = '';

    this.gameLogs.forEach(log => {
      const tr = document.createElement('tr');
      tr.innerHTML = `
        <td>${log.time}</td>
        <td>${log.name} (${log.class})</td>
        <td>${log.level}</td>
        <td>${log.score}</td>
        <td>${log.accuracy}</td>
        <td>${log.replays}</td>
      `;
      tbody.appendChild(tr);
    });

    document.getElementById('teacher-modal').style.display = 'flex';
  }

  exportCSV() {
    if (this.gameLogs.length === 0) {
      alert("No student data logs available yet!");
      return;
```

```
    }

    let csvContent = "data:text/csv;charset=utf-8,Timestamp,Student
Name,Class,Difficulty,Score,Accuracy,Audio Replays\n";
    this.gameLogs.forEach(l => {
      csvContent += `"${l.time}","${l.name}","${l.class}","${l.level}",${l.score},"$
{l.accuracy}",${l.replays}\n`;
    });

    const encodedUri = encodeURI(csvContent);
    const link = document.createElement("a");
    link.setAttribute("href", encodedUri);
    link.setAttribute("download", `Pizzeria_Listening_Results_$
{this.studentName.replace(/\s+/g, '_')}.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  }

  syncToGoogleSheets() {
    const webhookUrl = document.getElementById('webhook-url').value;
    if (!webhookUrl) {
      alert("Please enter a valid Google Apps Script Webhook URL!");
      return;
    }

    fetch(webhookUrl, {
      method: 'POST',
      mode: 'no-cors',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(this.gameLogs)
    }).then(() => alert("Data synced to Google Sheets!"))
      .catch(err => alert("Error syncing data: " + err));
  }

  /* Render Loop */
  animate() {
    requestAnimationFrame(() => this.animate());

    // Player Movement Logic
    const speed = 0.08;
    let moveX = 0, moveZ = 0;

    if (this.keys['w'] || this.keys['arrowup']) moveZ -= speed;
    if (this.keys['s'] || this.keys['arrowdown']) moveZ += speed;
```

```
    if (this.keys['a'] || this.keys['arrowleft']) moveX -= speed;
    if (this.keys['d'] || this.keys['arrowright']) moveX += speed;

    // Apply Touch Vector
    moveX += this.moveVector.x * speed;
    moveZ += this.moveVector.z * speed;

    // Apply Rotation to Movement Vector
    const cosY = Math.cos(this.yaw);
    const sinY = Math.sin(this.yaw);

    this.camera.position.x += moveX * cosY + moveZ * sinY;
    this.camera.position.z += -moveX * sinY + moveZ * cosY;

    // Bounds Clamp
    this.camera.position.x = Math.max(-5, Math.min(5, this.camera.position.x));
    this.camera.position.z = Math.max(-5, Math.min(5, this.camera.position.z));

    // Camera Orientation
    const euler = new THREE.Euler(this.pitch, this.yaw, 0, 'YXZ');
    this.camera.quaternion.setFromEuler(euler);

    // Highlight Crosshair when facing stations
    const crosshair = document.getElementById('crosshair');
    let facingStation = false;
    Object.values(this.stations).forEach(st => {
      if (this.camera.position.distanceTo(st.position) < 2.5) facingStation = true;
    });
    if (facingStation) crosshair.classList.add('active');
    else crosshair.classList.remove('active');

    this.renderer.render(this.scene, this.camera);
  }

  onWindowResize() {
    this.camera.aspect = window.innerWidth / window.innerHeight;
    this.camera.updateProjectionMatrix();
    this.renderer.setSize(window.innerWidth, window.innerHeight);
  }
}

// Initialize Game Instance
let game;
window.addEventListener('DOMContentLoaded', () => {
  game = new PizzeriaGame();
```

```
    });
  </script>
</body>
</html>
```